# Regular Expressions

```
- Find a pattern in text file(s).
- Regular expressions are patterns used to match character combinations in
strings.
- A regular expression pattern is composed of simple characters, or a combination
of simple and special characters e.g. /abc/, /ab*c/
```

## grep

- Regex commands (Regular Expressions)
    - grep - GNU Regular Expression Parser
        - Basic wild-card
    - egrep - Extended Grep
        - Basic + Extended wild-card
    - fgrep - Fixed Grep
        - No wild-card
- Command syntax
  - grep "pattern" filepath
  - grep [options] "pattern" filepath
  - -c : count number of occurrences
  - -v : invert the find output
  - -i : case insensitive search
  - -w : search whole words only
  - -R : search recursively in a directory
  - -n : show line number
  -
- Pattern is given using regex wild-card characters.
    - Basic wild-card characters
        - `$` - find at the end of line.
        - `^` - find at the start of line.
        - `[ ]` - any single char in give range or set of chars
        - `[^ ]` - any single char not in give range or set of chars
        - `.` - any single character
        - `*` - zero or more occurrences of previous character
- Extended wild-card characters
    - `?` - zero or one occurrence of previous character
    - `+` - one or more occurrences of previous character
    - `{n}` - n occurrences of previous character
    - `{,n}` - max n occurrences of previous character

  - `{m,}` - min m occurrences of previous character
  - `{m,n}` - min m and max n occurrences of previous character
  - `()` - grouping (chars)
  - `(|)` - find one of the group of characters

---

- Anchoring - `^, $`
- Single Character Matching - `.`
- Bracket Expressions - `[], [^]`
- Quantifiers - `*, ?, +, {n}, {n,}, {,n}, {m,n}`
- Alternation - `|`
- Grouping - `()`

---

- man grep
- cat > taste.txt

```
These biscuits are good but cake is better
I think cake is good for health
Is like to eat cake

Biscuits are not good for health
These is  a good day Is
is
```.

- cat taste.txt
- grep "good" taste.txt : highlight good part
- grep "is" taste.txt : highlight is part
- grep -n "is" taste.txt : provide matching line num
- grep -w "is" taste.txt : find sep word
- grep -v "is" taste.txt : shows those where pattern not present
- grep -i "Is" taste.txt : ignore case sensitivity
- grep -c "is" taste.txt : give the count how many time patt occur
- grep "^is" taste.txt :find line whose start with pattern
- grep "is$" taste.txt :find line whose end with pattern
- grep "^is$" taste.txt :find line whose start and end with pattern (means single word)
- grep "^$" taste.txt : Matches empty lines

- 
- cat >> taste.txt
- cat taste.txt
- grep -n "^$" taste.txt :Matches empty lines with line num
- grep "is|good" taste.txt
    - This won't work as expected in basic grep.
    - is not recognized as "OR" in basic regex — it's treated as a literal character.
- egrep "is|good" taste.txt
    - This does work in basic grep because of the escaped |, which now means OR.
    - Matches lines containing either:
- cat > pattern.txt
- ```
  dc
  dac
  dec
  duc
  dmc
  doc
  d*c
  dxyc
  etc
  c
  ac
  cc
  ant
  ```
- grep "dc" pattern.txt : Matches: Any line containing exactly dc
- grep "dac" pattern.txt : Matches: Any line containing exactly dac
- ** here . => means any one char match in between **
- grep "d.c" pattern.txt : print those who start with d and c
- grep "d..c" pattern.txt :
    - Two wildcards: .. = two of any characters
    - start with d in ber 2anychar end with c
- grep "d[ae]c" pattern.txt
    - :Character class: matches either a or e
    - Matches: dac or dec
  ```

- grep "d[aem]c" pattern.txt
    - Character class: matches a, e, or m
    - Matches: dac, dec, or dmc
- grep "d[aem*]c" pattern.txt :
    - d followed by one character from set a, e, m, *, then c
    - So it matches: dac, dec, d*c, etc.
    - Note: * inside [] is treated as a literal character, not a wildcard.
- grep "d[^aem*]c" pattern.txt :[^...]
  - NOT match [a, e, m, or *] in between d and c
  - Matches: Any line where d is followed by a character other than a, e, m, or *, and then c
  -
- grep "d[*]c" pattern.txt
    - *[] = Match literal * , Matches: dc*
- grep "d*c" pattern.txt
    - In basic grep, * means "zero or more of the previous character"
    - So: d*c = zero or more d char followed by c
    - Matches: c, dc, ddc, dddc, etc.
    - (simple here it find those word whose end with c)
    - (d*c previous character before * or c any match)
- grep "d*c" pattern.txt :
    - grep "d*c" pattern.txt
    - Escapes the * to match literal *
    - Matches: d*c
- grep -F "d*c" pattern.txt
    - : -F = Fixed string search (no regex)
    - Treats everything literally
    - Matches: d*c exactly
- fgrep "d*c*" *pattern.txt :*
  - *Equivalent to: grep -F "d*c*" pattern.txt*
  - fgrep is the deprecated name for "fixed grep", which searches for literal strings, not regex.
  - Matches the exact text d*c in pattern.txt
  - It does not treat * as a special character (wildcard) — it's just a plain *.
- grep "d*c" pattern.txt
    - * means: 0 or more of the previous character
    - But here, it's d* → zero or more ds, followed by c
    - Matches: c, dc, ddc, dddc, etc.

- grep "da*c" pattern.txt
    - a* → zero or more as between d and c
    - Matches: dc, dac, daac, daaaac, etc.
    - Will NOT match ddac or dbc

cat > repeat.txt

```
dc
dac
daac
daaaac
daaaaac
```

- egrep "da?c" repeat.txt :
    - a? = zero or one a
    - Matches:dc,dac
- egrep "da+c" repeat.txt :
    - a+ = one or more a
    - Matches: dac,daac,daaaac,daaaaac
    - not match with: dc (because no a)
- egrep "da{2}c" repeat.txt :
    - a{2} = exactly 2 as
    - Matches: daac
- egrep "da{4}c" repeat.txt :
    - a{4} = exactly 2 as
    - Matches: daaaac
- egrep "da{2,4}c" repeat.txt :
    - a{2,4} = between 2 and 4 as
    - Matches: daac,daaaac,daaaaac
- egrep "da{2,}c" repeat.txt
    - :a{2,} = 2 or more as
    - Matches: daac ,daaaac ,daaaaac
- egrep "da{,4}c" repeat.txt :
    - maximum 4->a allow in between d and c
- egrep "dc" repeat.txt :Exact match: dc
- egrep "dac" repeat.txt :Exact match: dac

- egrep "daac" repeat.txt :Exact match: daac
- egrep "daa{1}c" repeat.txt :
    - a{1} = exactly 1 a
    - So this is same as: daac
- egrep "d(aa){1}c" repeat.txt
    - :Grouping: exactly one repetition of aa So pattern is daac
    - Matches: daac
- egrep "d(aa){2}c" repeat.txt
    - Group aa repeated 2 times = daaaac